# PRODUCT REQUIREMENTS DOCUMENT (PRD)

EduTrack Academy Management System Engine

**Document Version:** 1.0 **Target Architecture:** Python 3.x Standard Library Stack **Deployment Model:** CLI Local Runtime

## 1. Executive Summary & Project Goal

**EduTrack** is an industry-grade, terminal-based local Academy Management Engine engineered for administrators to handle student registries, transactional unique attendance logs, academic course assignments, and specialized multifaceted grade matrices.

The primary software engineering objective is to deliver a lightweight, secure, and production-ready local application utilizing strictly defensive architectural patterns. By executing this project relying entirely on the native **Python Standard Library**, it systematically demonstrates engineering mastery over programmatic workflows, computer logic compilation, and state serialization without hiding behind heavy third-party framework abstractions (such as Django, FastAPI, or external SQL database servers).

## 2. Technical Stack & Environment Requirements

- **Language Environment:** Native Python 3.8 or higher interpreter environments.
- **Storage Subsystem:** Local decoupled Flat-File Document Storage utilizing JSON serialization architecture (persistent file: `database.json`).
- **Internal Dependencies:** Strict encapsulation of core standard sub-modules: `json`, `datetime`, `os`, `unittest`.
- **External Ecosystem Dependencies:** 0% allocation. Zero external `pip` third-party dependencies are required for compilation.

## 3. System Architecture & Syllabus Domain Mapping

To guarantee baseline architectural integrity, every technical segment covered within advanced programming syllabus frameworks is structurally integrated and mapped directly to concrete functional assets within the engine:

| Core Python Concept | Architectural Requirement Implementation |
| --- | --- |
| **Object-Oriented Programming (OOP)** | Instantiation of an explicit base abstract data entity titled `User`, structurally inherited by a specialized child data object entity titled `Student`. |
| **Strict Data Encapsulation** | Privacy configurations assigned to internal student grade matrices via double-dunder notations (`__grades`), exposed safely only through localized transactional mutation validation criteria. |
| **Polymorphism** | Overriding behavioral interface routines (such as profile summary rendering via `get_dashboard()`) cleanly across parent and child subclass layers. |
| **Data Collections Architecture** | Utilizing linear `lists` for predictable sequential course tracks, associative map `dictionaries` for key-value performance matrices, and mathematical `sets` to naturally exclude data duplicate attendance flags. |
| **Advanced Meta-Decorators** | Programmatic execution of a custom closure-based `@log_action` structural wrapper to seamlessly inject timestamped security and audit logging without polluting pure business logic blocks. |
| **Functional Transformations** | Deployment of higher-order `map()` functions combined with clean inline `lambda` equations to compute system-wide curve alterations uniformly over datasets without state destruction. |
| **List Comprehensions** | High-efficiency single-line analytical vector operations designed to filter and isolate high-performing elements cleanly in memory. |
| **Generators & Iterators** | Building isolated memory pipeline streamers utilizing the `yield` directive to process heavy loops sequentially without triggering fatal heap memory allocation spikes. |
| **Defensive Error Management** | Encapsulating runtime lifecycles with robust `try...except...finally` structures to intercept user entry typing anomalies smoothly. |
| **Persistence Streams (File I/O)** | Automated system resource clean-up via context managers (`with open()`) translating state mutations back to static data layers upon user termination commands. |

# 4. Functional Specifications & Requirements

## 4.1 Data Modeling Layer

- **REQ-01 (Base Paradigm):** The core module must expose an foundational class container named `User` which maps structural arguments: `user_id`, `name`, and `email` uniformly.

- **REQ-02 (Inheritance Strategy):** The engine must extend the `User` profile blueprint into a `Student` subclass that scales data capability to provision tracked lists for course strings, state-based unique sets for calendar dates, and complex mapping records for performance logs.
- **REQ-03 (Encapsulated Restriction):** Direct state mutation of `__grades` from external namespaces is explicitly banned. Alterations must trigger internal setter checks which evaluate that input parameters conform precisely to real number scales between 0.0 and 100.0, throwing a descriptive `ValueError` on constraint violation.

### 4.2 Analytical & Algorithmic Computations

- **REQ-04 (Polymorphic View Override):** Invoking `get_dashboard()` across a Student pointer must seamlessly incorporate the base output data block derived from the superclass while dynamically parsing and appending unique structural attributes like enrollment lists, unique attendance metrics, and calculated GPAs.
- **REQ-05 (Mathematical Grade Mapping):** The internal engine must cleanly step through the private structural dictionaries, average all numerical test points, and accurately cast the resulting quotient onto an industry-standard 4.0 scaling matrix:

$$GPA = (\Sigma \text{ Scores} / \text{Total Count}) / 100 \times 4.0$$

- **REQ-06 (Functional Linear Scaling):** The engine must provide interfaces allowing admins to update scoring structures via functional programming chains using native `map()` and an inline evaluation `lambda` logic statement capped safely at 100.0.
- **REQ-07 (List Comprehension Filter):** Isolate Registry Honors list records efficiently via clean list comprehensions targeting all objects maintaining a dynamic evaluation threshold of $GPA \geq 3.5$.

### 4.3 Auditing Layers & System Resilience

- **REQ-08 (Auditing Wrapper):** Admin functions must accept wrapper intercepts using the `@log_action` decorator paradigm to capture universal parameter signatures (`*args`, `**kwargs`) and generate formatted timestamp logs inside the tracking stdout stream.
- **REQ-09 (Runtime Interception Boundary):** The primary command loop must encapsulate operational prompts inside explicit error traps to cleanly manage data formatting mismatches, missing search keys, or null input arguments without causing terminal stack termination.

### 4.4 File Lifecycle & Persistence Streams

- **REQ-10 (Generator Memory Pipeline):** Streaming operational profiles out of registries for large batch analytical processing must use structured iterator generators running on the `yield` keyword.
- **REQ-11 (Initialization Verification):** On launch, the subsystem must automatically verify the path state of `database.json`. If validated, the loader must cleanly decode standard arrays and cast JSON sequences into accurate memory types (e.g., matching string arrays back into native Python `sets`).

- **REQ-12 (State Preservation):** Upon exiting, the context interface must completely capture current entity state configurations, marshal the parameters down into standard indented JSON, and successfully lock data down to permanent media safely.

## 5. Non-Functional Verification Matrix

| Verification ID | Testing Context | Expected Application Outcome |
|---|---|---|
| **VAL-01** | Malformed Grading Input | Passing non-numeric strings into grading blocks triggers a safe `ValueError` trap instead of crashing the process execution stack. |
| **VAL-02** | Attendance Duplicate Isolation | Submitting an identical date string duplicate to the attendance ledger records a single entry instance via the underlying mathematical `set` structure. |
| **VAL-03** | Algorithmic Calibration | Evaluating standard parameters of 100 and 50 points outputs an exact calibrated index matching a metric of `3.00 GPA`. |
| **VAL-04** | Graceful Serialization | Invoking system option termination routines forces clean data marshaling out to disk buffers, saving database configurations intact. |